

第 5 章

用户身份与文件权限

本章讲解了如下内容：

- 用户身份与能力；
- 文件权限与归属；
- 文件的特殊权限；
- 文件的隐藏权限；
- 文件访问控制列表；
- su 命令与 sudo 服务。

Linux 是一个多用户、多任务的操作系统，具有很好的稳定性与安全性，在幕后保障 Linux 系统安全的则是一系列复杂的配置工作。本章将详细讲解文件的所有者、所属组以及其他用户可对文件进行的读（r）、写（w）、执行（x）等操作，以及如何在 Linux 系统中添加、删除、修改用户账户信息。我们还可以使用 SUID、SGID 与 SBIT 特殊权限更加灵活地设置系统权限功能，来弥补对文件设置一般操作权限时所带来的不足。隐藏权限能够给系统增加一层隐形的防护层，让黑客最多只能查看关键日志信息，而不能进行修改或删除。而文件的访问控制列表（Access Control List, ACL）可以进一步让单一用户、用户组对单一文件或目录进行特殊的权限设置，让文件具有能满足工作需求的最小权限。本章最后还将讲解如何使用 su 命令与 sudo 服务让普通用户具备管理员的权限，不仅可以满足日常的工作需求，还可以确保系统的安全性。

5.1 用户身份与能力

设计 Linux 系统的初衷之一就是为了满足多个用户同时工作的需求，因此 Linux 系统必须具备很好的安全性。第 1 章在安装 RHEL 7 操作系统时，特别要求设置 root 管理员密码，这个 root 管理员就是存在于所有类 UNIX 系统中的超级用户。它拥有最高的系统所有权，能够管理系统的各项功能，如添加/删除用户、启动/关闭服务进程、开启/禁用硬件设备等。虽然以 root 管理员的身份工作时不会受到系统的限制，但俗语讲“能力越大，责任就越大”，因此一旦使用这个高能的 root 管理员权限执行了错误的命令可能会直接毁掉整个系统。使用与否，确实需要好好权衡一下。

在学习时是否要使用 root 管理员权限来控制整个系统呢？面对这个问题，网络上有很多文章建议以普通用户的身份来操作——这是一个更安全也更“无责任”的回答。今天，刘遑老师就要冒天下之大不韪给出自己的心得——强烈推荐大家在学习时使用 root 管理员权限！

这种为 root 管理员正名的决绝态度在网络中应该还是很少见的，我之所以力荐 root 管理员权限，原因很简单。因为在 Linux 的学习过程中如果使用普通用户身份进行操作，则在配置服务之后出现错误时很难判断是系统自身的问题还是因为权限不足而导致的；这无疑会给大家的学习过程徒增坎坷。更何况我们的实验环境是使用 VMware 虚拟机软件搭建的，可以将安装好的系统设置为一次快照，这即便系统彻底崩溃了，您也可以在 5 秒钟的时间内快速还原出一台全新的系统，而不用担心数据丢失。

总之，刘遄老师在培训时都推荐每位学生使用 root 管理员权限来学习 Linux 系统，等到工作时再根据生产环境决定使用哪个用户权限；这些仅与选择相关，而非技术性问题。

另外，很多图书或培训机构的老师会讲到，Linux 系统中的管理员就是 root。这其实是错误的，Linux 系统的管理员之所以是 root，并不是因为它的名字叫 root，而是因为该用户的身份号码即 UID（User IDentification）的数值为 0。在 Linux 系统中，UID 就相当于我们的身份证号码一样具有唯一性，因此可通过用户的 UID 值来判断用户身份。在 RHEL 7 系统中，用户身份有下面这些。

- 管理员 UID 为 0：系统的管理员用户。
- 系统用户 UID 为 1~999：Linux 系统为了避免因某个服务程序出现漏洞而被黑客提权至整台服务器，默认服务程序会有独立的系统用户负责运行，进而有效控制被破坏范围。
- 普通用户 UID 从 1000 开始：是由管理员创建的用于日常工作的用户。

需要注意的是，UID 是不能冲突的，而且管理员创建的普通用户的 UID 默认是从 1000 开始的（即使前面有闲置的号码）。

为了方便管理属于同一组的用户，Linux 系统中还引入了用户组的概念。通过使用用户组号码（GID，Group IDentification），我们可以把多个用户加入到同一个组中，从而方便为组中的用户统一规划权限或指定任务。假设有一个公司中有多个部门，每个部门中又有很多员工。如果只想让员工访问本部门内的资源，则可以针对部门而非具体的员工来设置权限。例如，可以通过对技术部门设置权限，使得只有技术部门的员工可以访问公司的数据库信息等。

另外，在 Linux 系统中创建每个用户时，将自动创建一个与其同名的基本用户组，而且这个基本用户组只有该用户一个人。如果该用户以后被归纳入其他用户组，则这个其他用户组称之为扩展用户组。一个用户只有一个基本用户组，但是可以有多个扩展用户组，从而满足日常的工作需要。

5.1.1 useradd 命令

useradd 命令用于创建新的用户，格式为“useradd [选项] 用户名”。

可以使用 useradd 命令创建用户账户。使用该命令创建用户账户时，默认的用户家目录会被存放在 /home 目录中，默认的 Shell 解释器为 /bin/bash，而且默认会创建一个与该用户同名的基本用户组。这些默认设置可以根据表 5-1 中的 useradd 命令参数自行修改。

表 5-1 useradd 命令中的用户参数以及作用

参数	作用
-d	指定用户的家目录（默认为/home/username）
-e	账户的到期时间，格式为 YYYY-MM-DD.
-u	指定该用户的默认 UID
-g	指定一个初始的用户基本组（必须已存在）
-G	指定一个或多个扩展用户组
-N	不创建与用户同名的基本用户组
-s	指定该用户的默认 Shell 解释器

下面我们创建一个普通用户并指定家目录的路径、用户的 UID 以及 Shell 解释器。在下面的命令中，请注意/sbin/nologin，它是终端解释器中的一员，与 Bash 解释器有着天壤之别。一旦用户的解释器被设置为 nologin，则代表该用户不能登录到系统中：

```
[root@linuxprobe ~]# useradd -d /home/linux -u 8888 -s /sbin/nologin linuxprobe
[root@linuxprobe ~]# id linuxprobe
uid=8888(linuxprobe) gid=8888(linuxprobe) groups=8888(linuxprobe)
```

5.1.2 groupadd 命令

groupadd 命令用于创建用户组，格式为“groupadd [选项] 群组名”。

为了能够更加高效地指派系统中各个用户的权限，在工作中常常会把几个用户加入到一个组里面，这样便可以针对一类用户统一安排权限。创建用户组的步骤非常简单，例如使用如下命令创建一个用户组 ronny：

```
[root@linuxprobe ~]# groupadd ronny
```

5.1.3 usermod 命令

usermod 命令用于修改用户的属性，格式为“usermod [选项] 用户名”。

前文曾反复强调，Linux 系统中的一切都是文件，因此在系统中创建用户也就是修改配置文件的过程。用户的信息保存在/etc/passwd 文件中，可以直接用文本编辑器来修改其中的用户参数项目，也可以用 usermod 命令修改已经创建的用户信息，诸如用户的 UID、基本/扩展用户组、默认终端等。usermod 命令的参数以及作用如表 5-2 所示。

表 5-2 usermod 命令中的参数及作用

参数	作用
-c	填写用户账户的备注信息
-d -m	参数-m 与参数-d 连用，可重新指定用户的家目录并自动把旧的数据转移过去
-e	账户的到期时间，格式为 YYYY-MM-DD
-g	变更所属用户组

-G	变更扩展用户组
续表	
参数	作用
-L	锁定用户禁止其登录系统
-U	解锁用户，允许其登录系统
-s	变更默认终端
-u	修改用户的 UID

大家不要被这么多参数吓坏了。我们先来看一下账户 `linuxprobe` 的默认信息：

```
[root@linuxprobe ~]# id linuxprobe
uid=1000(linuxprobe) gid=1000(linuxprobe) groups=1000(linuxprobe)
```

然后将用户 `linuxprobe` 加入到 `root` 用户组中，这样扩展组列表中则会出现 `root` 用户组的字样，而基本组不会受到影响：

```
[root@linuxprobe ~]# usermod -G root linuxprobe
[root@linuxprobe ~]# id linuxprobe
uid=1000(linuxprobe) gid=1000(linuxprobe) groups=1000(linuxprobe),0(root)
```

再来试试用 `-u` 参数修改 `linuxprobe` 用户的 UID 号码值。除此之外，我们还可以用 `-g` 参数修改用户的基本组 ID，用 `-G` 参数修改用户扩展组 ID。

```
[root@linuxprobe ~]# usermod -u 8888 linuxprobe
[root@linuxprobe ~]# id linuxprobe
uid=8888(linuxprobe) gid=1000(linuxprobe) groups=1000(linuxprobe),0(root)
```

5.1.4 passwd 命令

`passwd` 命令用于修改用户密码、过期时间、认证信息等，格式为 “`passwd` [选项] [用户名]”。

普通用户只能使用 `passwd` 命令修改自身的系统密码，而 `root` 管理员则有权修改其他所有人的密码。更酷的是，`root` 管理员在 Linux 系统中修改自己或他人的密码时不需要验证旧密码，这一点特别方便。既然 `root` 管理员可以修改其他用户的密码，就表示完全拥有该用户的管理权限。`passwd` 命令中可用的参数以及作用如表 5-3 所示。

表 5-3 passwd 命令中的参数以及作用

参数	作用
-l	锁定用户，禁止其登录
-u	解除锁定，允许用户登录
--stdin	允许通过标准输入修改用户密码，如 <code>echo "NewPassWord" passwd --stdin Username</code>
-d	使该用户可用空密码登录系统
-e	强制用户在下次登录时修改密码
-S	显示用户的密码是否被锁定，以及密码所采用的加密算法名称

接下来刘遑老师将演示如何修改用户自己的密码，以及如何修改其他人的密码（修改他人密码时，需要具有 root 管理员权限）：

```
[root@linuxprobe ~]# passwd
Changing password for user root.
New password: 此处输入密码值
Retype new password: 再次输入进行确认
passwd: all authentication tokens updated successfully.
[root@linuxprobe ~]# passwd linuxprobe
Changing password for user linuxprobe.
New password: 此处输入密码值
Retype new password: 再次输入进行确认
passwd: all authentication tokens updated successfully.
```

假设您有位同事正在度假，而且假期很长，那么可以使用 `passwd` 命令禁止该用户登录系统，等假期结束回归工作岗位时，再使用该命令允许用户登录系统，而不是将其删除。这样既保证了这段时间内系统的安全，也避免了频繁添加、删除用户带来的麻烦：

```
[root@linuxprobe ~]# passwd -l linuxprobe
Locking password for user linuxprobe.
passwd: Success
[root@linuxprobe ~]# passwd -S linuxprobe
linuxprobe LK 2017-12-26 0 99999 7 -1 (Password locked.)
[root@linuxprobe ~]# passwd -u linuxprobe
Unlocking password for user linuxprobe.
passwd: Success
[root@linuxprobe ~]# passwd -S linuxprobe
linuxprobe PS 2017-12-26 0 99999 7 -1 (Password set, SHA512 crypt.)
```

5.1.5 userdel 命令

`userdel` 命令用于删除用户，格式为“`userdel [选项] 用户名`”。

如果我们确认某位用户后续不再会登录到系统中，则可以通过 `userdel` 命令删除该用户的所有信息。在执行删除操作时，该用户的家目录默认会保留下来，此时可以使用 `-r` 参数将其删除。`userdel` 命令的参数以及作用如表 5-4 所示。

表 5-4 userdel 命令的参数以及作用

参数	作用
<code>-f</code>	强制删除用户
<code>-r</code>	同时删除用户及用户家目录

下面使用 `userdel` 命令将 `linuxprobe` 用户删除，其操作如下：

```
[root@linuxprobe ~]# id linuxprobe
uid=8888(linuxprobe) gid=1000(linuxprobe) groups=1000(linuxprobe),0(root)
[root@linuxprobe ~]# userdel -r linuxprobe
[root@linuxprobe ~]# id linuxprobe
id: linuxprobe: no such user
```

5.2 文件权限与归属

尽管在 Linux 系统中一切都是文件，但是每个文件的类型不尽相同，因此 Linux 系统使用了不同的字符来加以区分，常见的字符如下所示。

- -: 普通文件。
- d: 目录文件。
- l: 链接文件。
- b: 块设备文件。
- c: 字符设备文件。
- p: 管道文件。

在 Linux 系统中，每个文件都有所属的所有者和所有组，并且规定了文件的所有者、所有组以及其他人对文件所拥有的可读（r）、可写（w）、可执行（x）等权限。对于一般文件来说，权限比较容易理解：“可读”表示能够读取文件的实际内容；“可写”表示能够编辑、新增、修改、删除文件的实际内容；“可执行”则表示能够运行一个脚本程序。但是，对于目录文件来说，理解其权限设置来就不那么容易了。很多资深 Linux 用户其实也没有真正搞明白。

刘遄老师在这里给大家详细讲解一下目录文件的权限设置。对目录文件来说，“可读”表示能够读取目录内的文件列表；“可写”表示能够在目录内新增、删除、重命名文件；而“可执行”则表示能够进入该目录。

文件的读、写、执行权限可以简写为 `rw`，亦可分别用数字 4、2、1 来表示，文件所有者、所属组及其他用户权限之间无关联，如表 5-5 所示。

表 5-5 文件权限的字符与数字表示

权限分配	文件所有者			文件所属组			其他用户		
权限项	读	写	执行	读	写	执行	读	写	执行
字符表示	r	w	x	r	w	x	r	w	x
数字表示	4	2	1	4	2	1	4	2	1

文件权限的数字法表示基于字符表示（`rw`）的权限计算而来，其目的是简化权限的表示。例如，若某个文件的权限为 `7` 则代表可读、可写、可执行（ $4+2+1$ ）；若权限为 `6` 则代表可读、可写（ $4+2$ ）。我们来看这样一个例子。现在有这样一个文件，其所有者拥有可读、可写、可执行的权限，其文件所属组拥有可读、可写的权限；而且其他人只有可读的权限。那么，这个文件的权限就是 `rw-rw-r--`，数字法表示即为 `764`。不过大家千万别再将这三个数字相加，计算出 $7+6+4=17$ 的结果，这是小学的数学加减法，不是 Linux 系统的权限数字表示法，三者之间没有互通关系。

Linux 系统的文件权限相当复杂，但是用途很广泛，建议大家把它彻底搞清楚之后再学习下一节的内容。现在来练习一下。请各位读者分别计算数字表示法 `764`、`642`、`153`、`731` 所对应的字符表示法，然后再把 `rw-rw-r--`、`rw--w--wx`、`rw-r--r--` 转换成数字表示法。

下面我们利用上文讲解的知识，一起分析图 5-1 中所示的文件信息。



图 5-1 通过 ls 命令查看到的文件属性信息

在图 5-1 中, 包含了文件的类型、访问权限、所有者 (属主)、所属组 (属组)、占用的磁盘大小、修改时间和文件名称等信息。通过分析可知, 该文件的类型为普通文件, 所有者权限为可读、可写 (rw-), 所属组权限为可读 (r--), 除此以外的其他人也只有可读权限 (r--), 文件的磁盘占用大小是 34298 字节, 最近一次的修改时间为 4 月 2 日的凌晨 23 分, 文件的名称为 install.log。

5.3 文件的特殊权限

在复杂多变的生产环境中, 单纯设置文件的 rwx 权限无法满足我们对安全和灵活性的需求, 因此便有了 SUID、SGID 与 SBIT 的特殊权限位。这是一种对文件权限进行设置的特殊功能, 可以与一般权限同时使用, 以弥补一般权限不能实现的功能。下面具体解释这 3 个特殊权限位的功能以及用法。

5.3.1 SUID

SUID 是一种对二进制程序进行设置的特殊权限, 可以让二进制程序的执行者临时拥有属主的权限 (仅对拥有执行权限的二进制程序有效)。例如, 所有用户都可以执行 passwd 命令来修改自己的用户密码, 而用户密码保存在 /etc/shadow 文件中。仔细查看这个文件就会发现它的默认权限是 000, 也就是说除了 root 管理员以外, 所有用户都没有查看或编辑该文件的权限。但是, 在使用 passwd 命令时如果加上 SUID 特殊权限位, 就可让普通用户临时获得程序所有者的身份, 把变更的密码信息写入到 shadow 文件中。这很像我们在古装剧中见到的手持尚方宝剑的钦差大臣, 他手持的尚方宝剑代表的是皇上的权威, 因此可以惩戒贪官, 但这并不意味着他永久成为了皇上。因此这只是一种有条件的、临时的特殊权限授权方法。

查看 passwd 命令属性时发现所有者的权限由 rwx 变成了 rws, 其中 x 改变成 s 就意味着该文件被赋予了 SUID 权限。另外有读者会好奇, 那么如果原本的权限是 rw- 呢? 如果原先权限位上没有 x 执行权限, 那么被赋予特殊权限后将变成大写的 S。

```
[root@linuxprobe ~]# ls -l /etc/shadow
----- 1 root root 1004 Jan 3 06:23 /etc/shadow
[root@linuxprobe ~]# ls -l /bin/passwd
-rwsr-xr-x 1 root root 27832 Jan 29 2017 /bin/passwd
```

5.3.2 SGID

SGID 主要实现如下两种功能:

- 让执行者临时拥有属组的权限 (对拥有执行权限的二进制程序进行设置);

➤ 在某个目录中创建的文件自动继承该目录的用户组（只可以对目录进行设置）。

SGID 的第一种功能是参考 SUID 而设计的，不同点在于执行程序的用户获取的不再是文件所有者的临时权限，而是获取到文件所属组的权限。举例来说，在早期的 Linux 系统中，`/dev/kmem` 是一个字符设备文件，用于存储内核程序要访问的数据，权限为：

```
cr--r----- 1 root system 2, 1 Feb 11 2017 kmem
```

大家看出问题了吗？除了 root 管理员或属于 system 组成员外，所有用户都没有读取该文件的权限。由于在平时我们需要查看系统的进程状态，为了能够获取到进程的状态信息，可在用于查看系统进程状态的 ps 命令文件上增加 SGID 特殊权限位。查看 ps 命令文件的属性信息：

```
-r-xr-sr-x 1 bin system 59346 Feb 11 2017 ps
```

这样一来，由于 ps 命令被增加了 SGID 特殊权限位，所以当用户执行该命令时，也就临时获取到了 system 用户组的权限，从而可以顺利地读取设备文件了。

前文提到，每个文件都有其归属的所有者和所属组，当创建或传送一个文件后，这个文件就会自动归属于执行这个操作的用户（即该用户是文件的所有者）。如果现在需要在一个部门内设置共享目录，让部门内的所有人员都能够读取目录中的内容，那么就可以创建部门共享目录后，在该目录上设置 SGID 特殊权限位。这样，部门内的任何人员在里面创建的任何文件都会归属于该目录的所属组，而不再是自己的基本用户组。此时，我们用到的就是 SGID 的第二个功能，即在某个目录中创建的文件自动继承该目录的用户组（只可以对目录进行设置）。

```
[root@linuxprobe ~]# cd /tmp
[root@linuxprobe tmp]# mkdir testdir
[root@linuxprobe tmp]# ls -ald testdir/
drwxr-xr-x. 2 root root 6 Feb 11 11:50 testdir/
[root@linuxprobe tmp]# chmod -Rf 777 testdir/
[root@linuxprobe tmp]# chmod -Rf g+s testdir/
[root@linuxprobe tmp]# ls -ald testdir/
drwxrwsrwx. 2 root root 6 Feb 11 11:50 testdir/
```

在使用上述命令设置好目录的 777 权限（确保普通用户可以向其中写入文件），并为该目录设置了 SGID 特殊权限位后，就可以切换至一个普通用户，然后尝试在该目录中创建文件，并查看新创建的文件是否会继承新创建的文件所在的目录的所属组名称：

```
[root@linuxprobe tmp]# su - linuxprobe
Last login: Wed Feb 11 11:49:16 CST 2017 on pts/0
[linuxprobe@linuxprobe ~]$ cd /tmp/testdir/
[linuxprobe@linuxprobe testdir]$ echo "linuxprobe.com" > test
[linuxprobe@linuxprobe testdir]$ ls -al test
-rw-rw-r--. 1 linuxprobe root 15 Feb 11 11:50 test
```

除了上面提到的 SGID 的这两个功能，我们再介绍两个与本小节内容相关的命令：chmod 和 chown。

chmod 命令是一个非常实用的命令，能够用来设置文件或目录的权限，格式为“chmod [参数] 权限 文件或目录名称”。如果要把一个文件的权限设置成其所有者可读可写可执行、所属组可读可写、其他人没有任何权限，则相应的字符法表示为 `rw-rw----`，其对应的数字法表示为 760。通过前面的基础学习和当前的练习实践，现在大家可以感受到使用数字法来设置文件权限的便捷性了吧。


```
[root@linuxprobe ~]# ls -al test
-rw-rw-r--. 1 linuxprobe root 15 Feb 11 11:50 test
[root@linuxprobe ~]# chmod 760 test
[root@linuxprobe ~]# ls -l test
-rwxrw----. 1 linuxprobe root 15 Feb 11 11:50 test
```

除了设置文件或目录的权限外，还可以设置文件或目录的所有者和所属组，这里使用的命令为 `chown`，其格式为“`chown [参数] 所有者:所属组 文件或目录名称`”。

`chmod` 和 `chown` 命令是用于修改文件属性和权限的最常用命令，它们还有一个特别的共性，就是针对目录进行操作时需要加上大写参数 `-R` 来表示递归操作，即对目录内所有的文件进行整体操作。

```
[root@linuxprobe ~]# ls -l test
-rwxrw----. 1 linuxprobe root 15 Feb 11 11:50 test
[root@linuxprobe ~]# chown root:bin test
[root@linuxprobe ~]# ls -l test
-rwxrw----. 1 root bin 15 Feb 11 11:50 test
```

5.3.3 SBIT

现在，大学里的很多老师都要求学生将作业上传到服务器的特定共享目录中，但总是有几个“破坏分子”喜欢删除其他同学的作业，这时就要设置 `SBIT`（`Sticky Bit`）特殊权限位了（也可以称之为特殊权限位之粘滞位）。`SBIT` 特殊权限位可确保用户只能删除自己的文件，而不能删除其他用户的文件。换句话说，当对某个目录设置了 `SBIT` 粘滞位权限后，那么该目录中的文件就只能被其所有者执行删除操作了。

最初不知道是哪位非资深技术人员将 `Sticky Bit` 直译成了“粘滞位”，刘遄老师建议将其称为“保护位”，这既好记，又能立刻让人了解它的作用。`RHEL 7` 系统中的 `/tmp` 作为一个共享文件的目录，默认已经设置了 `SBIT` 特殊权限位，因此除非是该目录的所有者，否则无法删除这里面的文件。

与前面所讲的 `SUID` 和 `SGID` 权限显示方法不同，当目录被设置 `SBIT` 特殊权限位后，文件的其他人权限部分的 `x` 执行权限就会被替换成 `t` 或者 `T`，原本有 `x` 执行权限则会写成 `t`，原本没有 `x` 执行权限则会被写成 `T`。

```
[root@linuxprobe tmp]# su - linuxprobe
Last login: Wed Feb 11 12:41:20 CST 2017 on pts/0
[linuxprobe@linuxprobe tmp]$ ls -ald /tmp
drwxrwxrwt. 17 root root 4096 Feb 11 13:03 /tmp
[linuxprobe@linuxprobe ~]$ cd /tmp
[linuxprobe@linuxprobe tmp]$ ls -ald
drwxrwxrwt. 17 root root 4096 Feb 11 13:03 .
[linuxprobe@linuxprobe tmp]$ echo "Welcome to linuxprobe.com" > test
[linuxprobe@linuxprobe tmp]$ chmod 777 test
[linuxprobe@linuxprobe tmp]$ ls -al test
-rwxrwxrwx. 1 linuxprobe linuxprobe 10 Feb 11 12:59 test
```

其实，文件能否被删除并不取决于自身的权限，而是看其所在目录是否有写入权限（其原理会在下个章节讲到）。为了避免现在很多读者不放心，所以上面的命令还是赋予了这个 `test`

文件最大的 777 权限（`rwrxrwx`）。我们切换到另外一个普通用户，然后尝试删除这个其他人创建的文件就会发现，即便读、写、执行权限全开，但是由于 SBIT 特殊权限位的缘故，依然无法删除该文件：

```
[root@linuxprobe tmp]# su - blackshield
Last login: Wed Feb 11 12:41:29 CST 2017 on pts/1
[blackshield@linuxprobe ~]$ cd /tmp
[blackshield@linuxprobe tmp]$ rm -f test
rm: cannot remove 'test': Operation not permitted
```

当然，要是也想对其他目录来设置 SBIT 特殊权限位，用 `chmod` 命令就可以了。对应的参数 `o+t` 代表设置 SBIT 粘滞位权限：

```
[blackshield@linuxprobe tmp]$ exit
Logout
[root@linuxprobe tmp]# cd ~
[root@linuxprobe ~]# mkdir linux
[root@linuxprobe ~]# chmod -R o+t linux/
[root@linuxprobe ~]# ls -ld linux/
drwxr-xr-t. 2 root root 6 Feb 11 19:34 linux/
```

5.4 文件的隐藏属性

Linux 系统中的文件除了具备一般权限和特殊权限之外，还有一种隐藏权限，即被隐藏起来的权限，默认情况下不能被用户发觉。有用户曾经在生产环境和 RHCE 考试题目中碰到过明明权限充足但却无法删除某个文件的情况，或者仅能在日志文件中追加内容而不能修改或删除内容，这在一定程度上阻止了黑客篡改系统日志的图谋，因此这种“奇怪”的文件也保障了 Linux 系统的安全性。

5.4.1 chattr 命令

`chattr` 命令用于设置文件的隐藏权限，格式为“`chattr[参数] 文件`”。如果想要把某个隐藏功能添加到文件上，则需要在命令后面追加“+参数”，如果想要把某个隐藏功能移出文件，则需要追加“-参数”。`chattr` 命令中可供选择的隐藏权限参数非常丰富，具体如表 5-6 所示。

表 5-6 `chattr` 命令中用于隐藏权限的参数及其作用

参数	作用
i	无法对文件进行修改；若对目录设置了该参数，则仅能修改其中的子文件内容而不能新建或删除文件
a	仅允许补充（追加）内容，无法覆盖/删除内容（Append Only）
S	文件内容在变更后立即同步到硬盘（sync）
s	彻底从硬盘中删除，不可恢复（用 0 填充原文件所在硬盘区域）
A	不再修改这个文件或目录的最后访问时间（atime）

续表

参数	作用
b	不再修改文件或目录的存取时间
D	检查压缩文件中的错误
d	使用 <code>dump</code> 命令备份时忽略本文件/目录
c	默认将文件或目录进行压缩
u	当删除该文件后依然保留其在硬盘中的数据，方便日后恢复
t	让文件系统支持尾部合并（ <code>tail-merging</code> ）
X	可以直接访问压缩文件中的内容

为了让读者能够更好地见识隐藏权限的效果，我们先来创建一个普通文件，然后立即尝试删除（这个操作肯定会成功）：

```
[root@linuxprobe ~]# echo "for Test" > linuxprobe
[root@linuxprobe ~]# rm linuxprobe
rm: remove regular file 'linuxprobe'? y
```

实践是检验真理的唯一标准。如果您没有亲眼见证过隐藏权限强大功能的美妙，就一定不会相信原来 Linux 系统会如此安全。接下来我们再次新建一个普通文件，并为其设置不允许删除与覆盖（+a 参数）权限，然后再尝试将这个文件删除：

```
[root@linuxprobe ~]# echo "for Test" > linuxprobe
[root@linuxprobe ~]# chmod +a linuxprobe
[root@linuxprobe ~]# rm linuxprobe
rm: remove regular file 'linuxprobe'? y
rm: cannot remove 'linuxprobe': Operation not permitted
```

可见，上述操作失败了。

5.4.2 lsattr 命令

`lsattr` 命令用于显示文件的隐藏权限，格式为“`lsattr [参数] 文件`”。在 Linux 系统中，文件的隐藏权限必须使用 `lsattr` 命令来查看，平时使用的 `ls` 之类的命令则看不出端倪：

```
[root@linuxprobe ~]# ls -al linuxprobe
-rw-r--r--. 1 root root 9 Feb 12 11:42 linuxprobe
```

一旦使用 `lsattr` 命令后，文件上被赋予的隐藏权限马上就会原形毕露。此时可以按照显示的隐藏权限的类型（字母），使用 `chattr` 命令将其去掉：

```
[root@linuxprobe ~]# lsattr linuxprobe
-----a----- linuxprobe
[root@linuxprobe ~]# chattr -a linuxprobe
[root@linuxprobe ~]# lsattr linuxprobe
----- linuxprobe
[root@linuxprobe ~]# rm linuxprobe
rm: remove regular file 'linuxprobe'? y
```

5.5 文件访问控制列表

不知道大家是否发现, 前文讲解的一般权限、特殊权限、隐藏权限其实有一个共性——权限是针对某一类用户设置的。如果希望对某个指定的用户进行单独的权限控制, 就需要用到文件的访问控制列表 (ACL) 了。通俗来讲, 基于普通文件或目录设置 ACL 其实就是针对指定的用户或用户组设置文件或目录的操作权限。另外, 如果针对某个目录设置了 ACL, 则目录中的文件会继承其 ACL; 若针对文件设置了 ACL, 则文件不再继承其所在目录的 ACL。

为了更直观地看到 ACL 对文件权限控制的强大效果, 我们先切换到普通用户, 然后尝试进入 root 管理员的家目录中。在没有针对普通用户对 root 管理员的家目录设置 ACL 之前, 其执行结果如下所示:

```
[root@linuxprobe ~]# su - linuxprobe
Last login: Sat Mar 21 16:31:19 CST 2017 on pts/0
[linuxprobe@linuxprobe ~]$ cd /root
-bash: cd: /root: Permission denied
[linuxprobe@linuxprobe root]$ exit
```

5.5.1 setfacl 命令

setfacl 命令用于管理文件的 ACL 规则, 格式为 “setfacl [参数] 文件名称”。文件的 ACL 提供的是在所有者、所属组、其他人的读/写/执行权限之外的特殊权限控制, 使用 setfacl 命令可以针对单一用户或用户组、单一文件或目录来进行读/写/执行权限的控制。其中, 针对目录文件需要使用 -R 递归参数; 针对普通文件则使用 -m 参数; 如果想要删除某个文件的 ACL, 则可以使用 -b 参数。下面来设置用户在 /root 目录上的权限:

```
[root@linuxprobe ~]# setfacl -Rm u:linuxprobe:rwX /root
[root@linuxprobe ~]# su - linuxprobe
Last login: Sat Mar 21 15:45:03 CST 2017 on pts/1
[linuxprobe@linuxprobe ~]$ cd /root
[linuxprobe@linuxprobe root]$ ls
anaconda-ks.cfg Downloads Pictures Public
[linuxprobe@linuxprobe root]$ cat anaconda-ks.cfg
[linuxprobe@linuxprobe root]$ exit
```

是不是觉得效果很酷呢? 但是现在有这样一个小问题——怎么去查看文件上有那些 ACL 呢? 常用的 ls 命令是看不到 ACL 表信息的, 但是却可以看到文件的权限最后一个点 (.) 变成了加号 (+), 这就意味着该文件已经设置了 ACL 了。现在大家是不是感觉学得越多, 越不敢说自己精通 Linux 系统了吧? 就这么一个不起眼的点 (.), 竟然还表示这么一种重要的权限。

```
[root@linuxprobe ~]# ls -ld /root
dr-xrwx---+ 14 root root 4096 May 4 2017 /root
```

5.5.2 getfacl 命令

getfacl 命令用于显示文件上设置的 ACL 信息, 格式为 “getfacl 文件名称”。Linux 系统

中的命令就是这么又可爱又好记。想要设置 ACL，用的是 `setfacl` 命令；要想查看 ACL，则用的是 `getfacl` 命令。下面使用 `getfacl` 命令显示在 `root` 管理员家目录上设置的所有 ACL 信息。

```
[root@linuxprobe ~]# getfacl /root
getfacl: Removing leading '/' from absolute path names
# file: root
# owner: root
# group: root
user::r-x
user:linuxprobe:rw
group::r-x
mask::rw
other::---
```

5.6 su 命令与 sudo 服务

各位读者在实验环境中很少遇到安全问题，并且为了避免因权限因素导致配置服务失败，从而建议使用 `root` 管理员来学习本书，但是在生产环境中还是要对安全多一份敬畏之心，不要用 `root` 管理员去做所有事情。因为一旦执行了错误的命令，可能会直接导致系统崩溃，这样一来，不但客户指责、领导批评，没准奖金也会鸡飞蛋打。但转头一想，尽管 Linux 系统为了安全性考虑，使得许多系统命令和服务只能被 `root` 管理员来使用，但是这也让普通用户受到了更多的权限束缚，从而导致无法顺利完成特定的工作任务。

`su` 命令可以解决切换用户身份的需求，使得当前用户在不退出登录的情况下，顺畅地切换到其他用户，比如从 `root` 管理员切换至普通用户：

```
[root@linuxprobe ~]# id
uid=0(root) gid=0(root) groups=0(root)
[root@linuxprobe ~]# su - linuxprobe
Last login: Wed Jan 4 01:17:25 EST 2017 on pts/0
[linuxprobe@linuxprobe ~]$ id
uid=1000(linuxprobe) gid=1000(linuxprobe) groups=1000(linuxprobe) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

细心的读者一定会发现，上面的 `su` 命令与用户名之间有一个减号 (-)，这意味着完全切换到新的用户，即把环境变量信息也变更为新用户的相应信息，而不是保留原始的信息。强烈建议在切换用户身份时添加这个减号 (-)。

另外，当从 `root` 管理员切换到普通用户时是不需要密码验证的，而从普通用户切换成 `root` 管理员就需要进行密码验证了；这也是一个必要的安全检查：

```
[linuxprobe@linuxprobe root]$ su root
Password:
[root@linuxprobe ~]# su - linuxprobe
Last login: Mon Aug 24 19:27:09 CST 2017 on pts/0
[linuxprobe@linuxprobe ~]$ exit
logout
[root@linuxprobe ~]#
```

尽管像上面这样使用 `su` 命令后，普通用户可以完全切换到 `root` 管理员身份来完成相应工作，

但这将暴露 root 管理员的密码，从而增大了系统密码被黑客获取的几率；这并不是最安全的方案。

刘遛老师接下来将介绍如何使用 sudo 命令把特定命令的执行权限赋予给指定用户，这样既可保证普通用户能够完成特定的工作，也可以避免泄露 root 管理员密码。我们要做的就是合理配置 sudo 服务，以便兼顾系统的安全性和用户的便捷性。sudo 服务的配置原则也很简单——在保证普通用户完成相应工作的前提下，尽可能少地赋予额外的权限。

sudo 命令用于给普通用户提供额外的权限来完成原本 root 管理员才能完成的任务，格式为“sudo [参数] 命令名称”。sudo 服务中可用的参数以及相应的作用如表 5-7 所示。

表 5-7 sudo 服务中的可用参数以及作用

参数	作用
-h	列出帮助信息
-l	列出当前用户可执行的命令
-u 用户名或 UID 值	以指定的用户身份执行命令
-k	清空密码的有效时间，下次执行 sudo 时需要再次进行密码验证
-b	在后台执行指定的命令
-p	更改询问密码的提示语

总结来说，sudo 命令具有如下功能：

- 限制用户执行指定的命令；
- 记录用户执行的每一条命令；
- 配置文件（/etc/sudoers）提供集中的用户管理、权限与主机等参数；
- 验证密码的后 5 分钟内（默认值）无须再让用户再次验证密码。

当然，如果担心直接修改配置文件会出现问题，则可以使用 sudo 命令提供的 visudo 命令来配置用户权限。这条命令在配置用户权限时将禁止多个用户同时修改 sudoers 配置文件，还可以对配置文件内的参数进行语法检查，并在发现参数错误时进行报错。

注：

只有 root 管理员才可以使用 visudo 命令编辑 sudo 服务的配置文件。

```
visudo: >>> /etc/sudoers: syntax error near line 111 <<<
What now?
Options are:
(e)dit sudoers file again
(x)it without saving changes to sudoers file
(Q)uit and save changes to sudoers file (DANGER!)
```

使用 visudo 命令配置 sudo 命令的配置文件时，其操作方法与 Vim 编辑器中用到的方法一致，因此在编写完成后记得在末行模式下保存并退出。在 sudo 命令的配置文件中，按照下面的格式将第 99 行（大约）填写上指定的信息：

谁可以使用 允许使用的主机=（以谁的身份） 可执行命令的列表

```
[root@linuxprobe ~]# visudo
96 ##
97 ## Allow root to run any commands anywhere
```

```
98 root ALL=(ALL) ALL
99 linuxprobe ALL=(ALL) ALL
```

在填写完毕后记得要先保存再退出,然后切换至指定的普通用户身份,此时就可以用 `sudo -l` 命令查看所有可执行的命令了(下面的命令中,验证的是该普通用户的密码,而不是 root 管理员的密码,请读者不要搞混了):

```
[root@linuxprobe ~]# su - linuxprobe
Last login: Thu Sep 3 15:12:57 CST 2017 on pts/1
[linuxprobe@linuxprobe ~]$ sudo -l
[sudo] password for linuxprobe:此处输入 linuxprobe 用户的密码
Matching Defaults entries for linuxprobe on this host:
requiretty, !visiblepw, always_set_home, env_reset, env_keep="COLORS
DISPLAY HOSTNAME HISTSIZE INPUTRC KDEDIR LS_COLORS", env_keep+="MAIL PS1
PS2 QTDIR USERNAME LANG LC_ADDRESS LC_CTYPE", env_keep+="LC_COLLATE
LC_IDENTIFICATION LC_MEASUREMENT LC_MESSAGES", env_keep+="LC_MONETARY
LC_NAME LC_NUMERIC LC_PAPER LC_TELEPHONE", env_keep+="LC_TIME LC_ALL
LANGUAGE LINGUAS _XKB_CHARSET XAUTHORITY",
secure_path=/sbin\:bin\:usr/sbin\:usr/bin
User linuxprobe may run the following commands on this host:
(ALL) ALL
```

接下来是见证奇迹的时刻! 作为一名普通用户, 是肯定不能看到 root 管理员的家目录 (/root) 中的文件信息的, 但是, 只需要在想执行的命令前面加上 `sudo` 命令就可以了:

```
[linuxprobe@linuxprobe ~]$ ls /root
ls: cannot open directory /root: Permission denied
[linuxprobe@linuxprobe ~]$ sudo ls /root
anaconda-ks.cfg Documents initial-setup-ks.cfg Pictures Templates
Desktop Downloads Music Public Videos
```

效果立竿见影! 但是考虑到生产环境中不允许某个普通用户拥有整个系统中所有命令的最高执行权(这也不符合前文提到的权限赋予原则, 即尽可能少地赋予权限), 因此 `ALL` 参数就有些不合适了。因此只能赋予普通用户具体的命令以满足工作需求, 这也受到了必要的权限约束。如果需要让某个用户只能使用 root 管理员的身份执行指定的命令, 切记一定要给出该命令的绝对路径, 否则系统会识别不出来。我们可以先使用 `whereis` 命令找出命令所对应的保存路径, 然后把配置文件第 99 行的用户权限参数修改成对应的路径即可:

```
[linuxprobe@linuxprobe ~]$ exit
logout
[root@linuxprobe ~]# whereis cat
cat: /usr/bin/cat /usr/share/man/man1/cat.1.gz /usr/share/man/man1p/cat.1p.gz
[root@linuxprobe ~]# visudo
96 ##
97 ## Allow root to run any commands anywhere
98 root ALL=(ALL) ALL
99 linuxprobe ALL=(ALL) /usr/bin/cat
```

在编辑好后依然是先保存再退出。再次切换到指定的普通用户, 然后尝试正常查看某个文件的内容, 此时系统提示没有权限。这时再使用 `sudo` 命令就可以顺利地查看文件内容了:

```
[root@linuxprobe ~]# su - linuxprobe
Last login: Thu Sep 3 15:51:01 CST 2017 on pts/1
```

```
[linuxprobe@linuxprobe ~]$ cat /etc/shadow
cat: /etc/shadow: Permission denied
[linuxprobe@linuxprobe ~]$ sudo cat /etc/shadow
root:$6$GV3UVtX4ZGg6ygA6$J9pBuPGUSgZslj83jyoI7ThJla9ZAULku3BcncAYF00Uwk6Sqc4E36
MnDlhLtlG9QadCpQCNVJs/5awHd0/pil:16626:0:99999:7:::
bin:!:16141:0:99999:7:::
daemon:!:16141:0:99999:7:::
adm:!:16141:0:99999:7:::
lp:!:16141:0:99999:7:::
sync:!:16141:0:99999:7:::
shutdown:!:16141:0:99999:7:::
halt:!:16141:0:99999:7:::
mail:!:16141:0:99999:7:::
operator:!:16141:0:99999:7:::
games:!:16141:0:99999:7:::
ftp:!:16141:0:99999:7:::
nobody:!:16141:0:99999:7:::
.....省略部分文件内容.....
```

大家千万不要以为到这里就结束了，刘遄老师还有更压箱底的宝贝。不知大家是否发觉在每次执行 `sudo` 命令后都会要求验证一下密码。虽然这个密码就是当前登录用户的密码，但是每次执行 `sudo` 命令都要输入一次密码其实也挺麻烦的，这时可以添加 `NOPASSWD` 参数，使得用户执行 `sudo` 命令时不再需要密码验证：

```
[linuxprobe@linuxprobe ~]$ exit
logout
[root@linuxprobe ~]# whereis poweroff
poweroff: /usr/sbin/poweroff /usr/share/man/man8/poweroff.8.gz
[root@linuxprobe ~]# visudo
.....省略部分文件内容.....
96 ##
97 ## Allow root to run any commands anywhere
98 root ALL=(ALL) ALL
99 linuxprobe ALL= NOPASSWD: /usr/sbin/poweroff
.....省略部分文件内容.....
```

这样，当切换到普通用户后再执行命令时，就不用再频繁地验证密码了，我们在日常工作中也就痛快至极了。

```
[root@linuxprobe ~]# su - linuxprobe
Last login: Thu Sep 3 15:58:31 CST 2017 on pts/1
[linuxprobe@linuxprobe ~]$ poweroff
User root is logged in on seat0.
Please retry operation after closing inhibitors and logging out other users.
Alternatively, ignore inhibitors and users with 'systemctl poweroff -i'.
[linuxprobe@linuxprobe ~]$ sudo poweroff
```

复习题

1. 在 RHEL 7 系统中，root 管理员是谁？

答：是 UID 为 0 的用户，默认是 root 管理员。

2. 如何使用 Linux 系统的命令行来添加或删除用户?

答: 添加和删除用户的命令分别是 `useradd` 与 `userdel`。

3. 若某个文件的所有者具有文件的读/写/执行权限, 其余人仅有读权限, 那么用数字法表示应该是什么?

答: 所有者权限为 `rw`, 所属组和其他人的权限为 `r--`, 因此数字法表示应该是 `744`。

4. 某链接文件的权限用数字法表示为 `755`, 那么相应的字符法表示是什么呢?

答: 在 Linux 系统中, 不同文件具有不同的类型, 因此这里应写成 `lrwxr-xr-x`。

5. 如果希望用户执行某命令时临时拥有该命令所有者的权限, 应该设置什么特殊权限?

答: 特殊权限中的 `SUID`。

6. 若对文件设置了隐藏权限 `+i`, 则意味着什么?

答: 无法对文件进行修改; 若对目录设置了该参数, 则仅能修改其中的子文件内容而不能新建或删除文件。

7. 使用访问控制列表 (ACL) 来限制 `linuxprobe` 用户组, 使得该组中的所有成员不得在 `/tmp` 目录中写入内容。

答: 想要设置用户组的 ACL, 则需要把 `u` 改成 `g`, 即 `setfacl -Rm g:linuxprobe:r-x /tmp`。

8. 当普通用户使用 `sudo` 命令时是否需要验证密码?

答: 系统在默认情况下需要验证当前登录用户的密码, 若不需要验证, 可添加 `NOPASSWD` 参数。